

# NOSCE(노스체)

-ai 심리적 힐링 서비스-

<이구(29)아나>

**<29아나> 러닝 페어 제안서**

✓ 일상 생활의 문제

- 현대인의 스트레스
- 지구 온난화
- 식단 관리에 대한 어려움
- OTT 과잉, 선택의 어려움
- 인강 선택의 어려움
- 시험 공부의 어려움
- 시간 관리의 어려움
- 저출산
- 고령화

✓ 문제 구분

| 컴퓨팅 기반 해결 가능 | 컴퓨팅 기반 해결 불가능 |
|---|---|
| - 현대인의 스트레스<br>- 식단 관리에 대한 어려움<br>- OTT 과잉, 선택의 어려움<br>- 인강 선택의 어려움<br>- 시험 공부의 어려움<br>- 시간 관리의 어려움 | - 지구 온난화<br>- 저출산<br>- 고령화 |

✓ 주제 선택

주제 선정 과정에서 일상 생활에서 겪는 다양한 문제들을 고민해보았습니다. 그 결과, 입시생, 취준생들이 겪는 시간 관리의 어려움, 바쁜 현대인들이 겪는 식단 관리의 어려움 등등, 사람들이 직면하고 있는 여러 가지 문제를 발견하게 되었습니다. 하지만, 시간 관리나 식단 관리와 같은 특정 분야의 문제보다는 더 근본적이고 더 광범위적인 심리적 도움에 초점을 맞추는 것이 중요하다고 생각했습니다. 현대 사회에서 많은 사람들이 겪는 스트레스와 불안, 그리고 정서적 고립은 단순한 시간 관리나 식단 관리로 해결될 수 있는 문제가 아니라고 생각했고, 따라서 저희는 사람들이 일상 속에서 심리적 안정과 회복력을 키울 수 있는 프로그램을 개발하고자 합니다.

✓ 해결 방안 고안

- 현대인들은 스트레스로 인해 생활에 있어 어려움을 겪지만 정신과를 가는 것은 기피하는 경향이 있습니다. 그렇기에 프로그램을 통해 자신의 심리 상태에 대한 자가진단하고, 후에 스트레스 해소에 도움을 줄 수 있는 문항들로 AI 친구 프로그램을 개발하여 보다 전문적으로, 또 친근하게 스트레스로 지친 현대인들에게 심리 치료 프로그램을 제공하는 해결 방안을 고안했습니다. 심리학에 근거하여 사용자의 심리 상태를 검진할 수 있는 문항들을 사용자에게 제시하면 사용자의 답변을 수치화해서 사용자의 심리 상태를 자가 진단 할 수 있도록 합니다. 자신의 심리 상태를 바탕으로 자신의 상태를 개선하기 위한 코멘트들을 AI 친구가 제안해주는 프로그램을 개발하는 것을 고안했습니다.
- 일상생활을 하면서 쌓이는 스트레스를 해결하기 위해서는 바쁜 현대인의 삶 속에서 소소하게 심리적 도움을 줄 수 있으면 좋겠다는 결론에 도달했습니다. 이에 따라 미술치료, 간단한 위로의 채팅, 나만의 소우주 꾸미기의 기능을 가진 프로그램을 개발하는 것을 고안해냈습니다. 이 프로그램은 먼저 사용자가 자신의 하루 일과에 대한 대화를 챗봇과 나눈 후에 나눈 대화를 기반으로 사용자의 심리에 도움이 되는 그림을 챗봇이 제안하여 사용자가 그림을 그리면 그 그림을 활용하여 프로그램 내의 사용자만의 소우주를 꾸며나가는 것입니다. 또한 챗봇이 사용자에게 간단한 위로의 채팅을 보냄으로써 딱딱하고 엄격한 이미지를 갖고 있는 기존의 챗봇으로부터 벗어나 사용자에게 따뜻하고 편안함을 주는 챗봇을 구현하려 합니다.
- 스트레스를 해결하기 위해서는 바쁜 현대인의 삶 속에서 짧게나마 누리는 휴식이 중요하다고 생각했습니다. 이 짧은 시간에 보다 효과적으로 휴식을 취하려면 명상을 하면 좋겠다는 아이디어가 떠올랐습니다. 그렇기에 챗봇이 추천한 노래에 맞춰 명상하는 시간을 갖도록 하는 프로그램을 고안해 보았습니다. 챗봇이 사용자에게 오늘의 기분에 대해 물어본 후 사용자가 자신의 기분을 작성하면 사용자의 기분에 따라 노래 영상을 유튜브 링크로 공유해주어서 그 노래에 맞춰 명상을 하도록 하는 프로그램입니다.

✓ 해결 방안 선택

현대 사회에서 많은 사람들이 겪는 스트레스와 불안, 그리고 정서적 고립에 심리적 도움을 주기 위한 방안을 고민해보았습니다. 이 때 전문적인 자격과 지식을 필요로 하는 의료적 도움은 주기 어렵다는 점에 유의하였습니다. 그 결과 해결방안으로 그림그리기를 통한 힐링, 이용자가 그린 그림으로 디지털 방 꾸미기, 음악추천, 심리적 위안을 제공하는 채팅기능, 명상법 알려주기, 호흡법 알려주기, 나무 키우기를 통한 힐링 등을 생각해보았습니다. 이 중 그림그리기를 통한 힐링, 이용자가 그린 그림으로 디지털 방 꾸미기, 심리적 위안을 제공하는 채

팅기능을 해결방안으로 선정하였습니다. 그 이유는 그림 그리기는 사용자가 자신의 감정과 생각을 시각적으로 표현할 수 있는 기회를 제공하기에 감정표현이 어려운 사람들도 쉽게 감정을 표현해 스트레스 해소를 통한 힐링을 얻을 수 있다고 생각하였기 때문입니다. 그리고 이용자가 그린 그림으로 디지털 방을 꾸미면 이용자가 성취감을 느낄 수 있고 프로그램 자체에 흥미를 느껴 그림그리기를 통해 감정표현하는 것에 더욱 도움을 줄 수 있다고 생각하였습니다. 그리고 단순히 자신의 감정을 표현하는 것뿐만 아니라 상호작용, 즉 위안을 받을 수도 있어야 한다고 생각되어 위안을 제공하는 채팅기능도 필요하다고 보았습니다. 반면에 음악추천과 명상법, 호흡법 알려주기의 경우 다른 검색 사이트를 통해 쉽게 얻을 수 있는 정보이기에 따로 기능을 제공할 필요가 없다고 생각했습니다. 마지막으로 나무 키우기의 경우 그림그리기와 마찬가지로 스트레스 완화와 힐링에 도움을 주는 것이 목적인데, 그림그리기에 비해 간접적이고 상징적인 표현이므로 즉각적인 감정 표현에 도움이 덜 될 수 있으며 그로 인해 스트레스 완화와 힐링효과가 그림그리기에 비해 덜할 것이라고 예상되었습니다. 또한, 그림 그리기는 사람마다 다르게 해석되고 표현될 수 있어, 개개인에게 맞춤형 경험을 제공할 수 있는 반면에 나무 키우기 서비스는 주어진 목표에 맞춰 진행되므로 더 구조적인 느낌이 들 수 있다고 생각되어 최종적으로 해결방안을 그림그리기를 통한 힐링, 이용자가 그린 그림으로 디지털 방 꾸미기, 심리적 위안을 제공하는 채팅기능으로 결정하게 되었습니다.

✓ 팀원 역할

| | |
|---|---|
| 유서현 (팀장) | 코딩, 자료조사, 주제고안, 해결 방안 선택 작성 |
| 김채현 | 코딩, 디자인 |
| 도건우 | 코딩, 자료조사, 주제선정과정과 이유 작성 |
| 양정현 | 코딩, 디자인, 문제 구분 작성 |
| 유서진 | 코딩, 자료조사, 주제고안, 해결 방안 고안 작성 |

# 〈29아나〉 러닝페어 중간 보고서

✓ 구현 방안

| Tkinter(파이썬 라이브러리) | Figma (UX/UI 디자인) | OpenAI API 활용 |
|---|---|---|

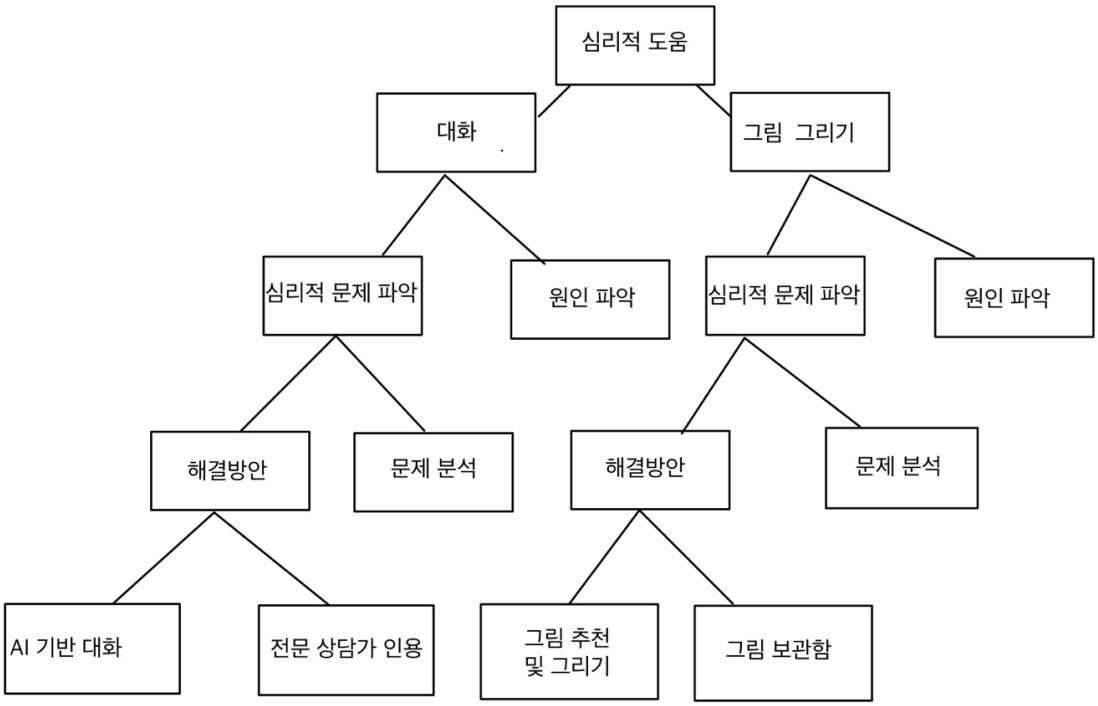
- Tkinter 라이브러리를 이용한 데스크탑 앱 개발을 목적으로 함
- UI/UX 디자인 프로그램 Figma를 이용하여 레이아웃 설계 및 디자인
- 대화 및 이미지 생성을 위해 OpenAI API 활용

(+) 선정 이유

- Tkinter: 조원 모두가 기본적으로 사용 가능.
- Figma: 웹 작업 형식, 공동 작업에 유리.

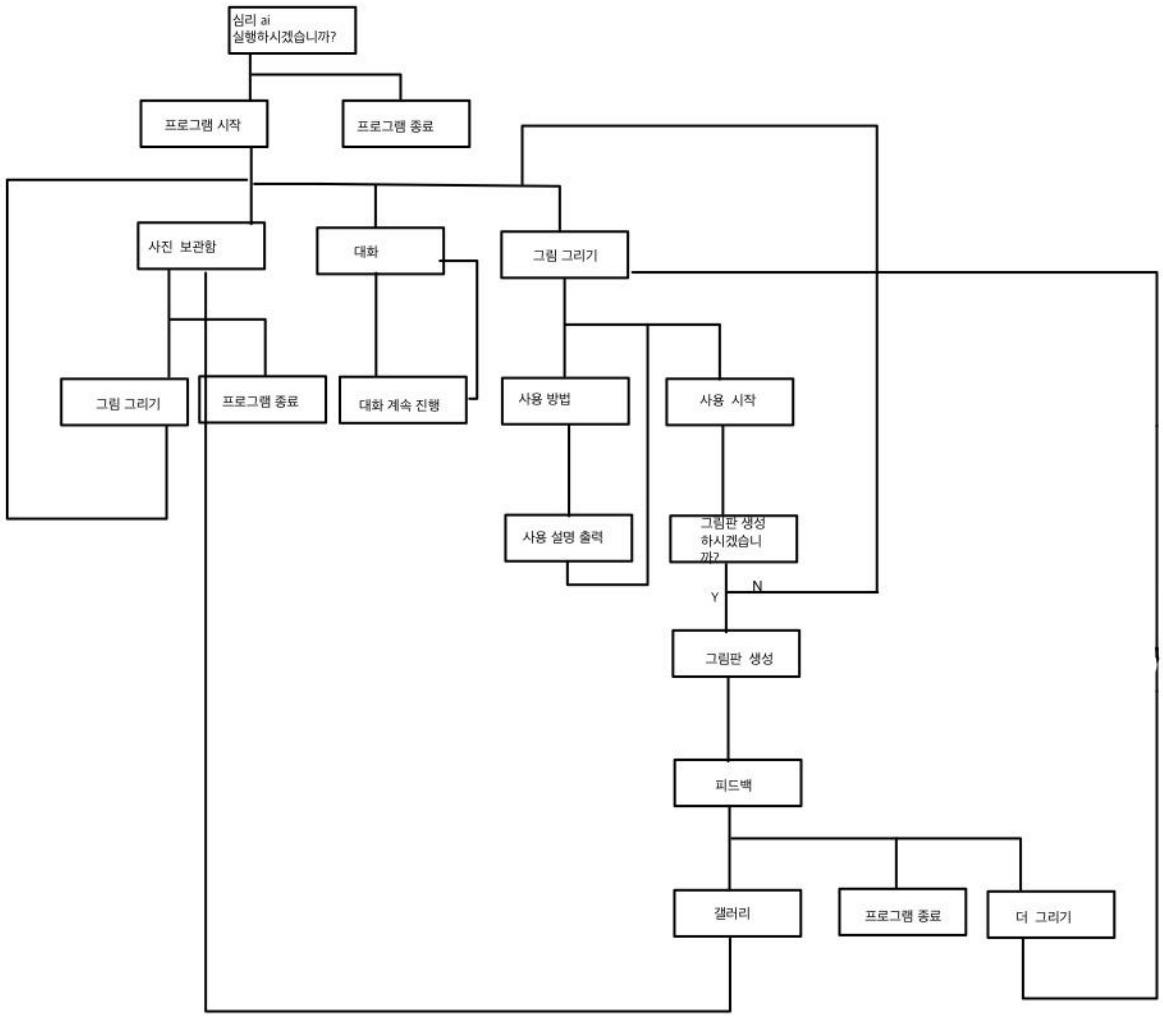
✓ 내용 구조

하행적 알고리즘 방식을 차용함.




(대화 방식의 구현 방향은 추후 고안할 예정)

✓ 알고리즘


심리 ai
실행하시겠습니까?
프로그램 시작
프로그램 종료
사진 보관함
대화
그림 그리기
그림 그리기
프로그램 종료
대화 계속 진행
사용 방법
사용 시작
사용 설명 출력
그림판 생성
하시겠습니
까?
Y
N
그림판 생성
피드백
갤러리
프로그램 종료
더 그리기

✓ 사용자 인터페이스

(대략적인 구조)

## 첫 방문 페이지 (1)




<기능>
- '-' 키 화면 최소화
- 'x' 키 화면 종료
- 빈칸에 이름 입력
- 'Enter' 키를 누르면 다음 화면으로 전환

<비고>
- 다른 기기에서도 정보가 유지될 필요가 없다고 판단하여 로그인 창을 만들지 않음
- 그 대신 이름을 입력받고, 이 화면부터 입력받은 정보들은 엑셀 파일을 생성, 정리하여 기기 내에서는 유지되도록 함

## 첫 방문 페이지 (2)




<기능>
- 대화 중: 입력된 이름 출력.
- '사용 방법' 키를 누르면 첫 방문 페이지 (3)으로 이동
- '사용 시작' 키를 누르면 기본 페이지 (1)로 이동

<비고>
- 채팅 형식이 아닌, 화면에 대화가 출력되는 형식
- 첫 화면에 API 낭비하지 않기 위해 단순 텍스트 출력

# 첫 방문 페이지 (3)



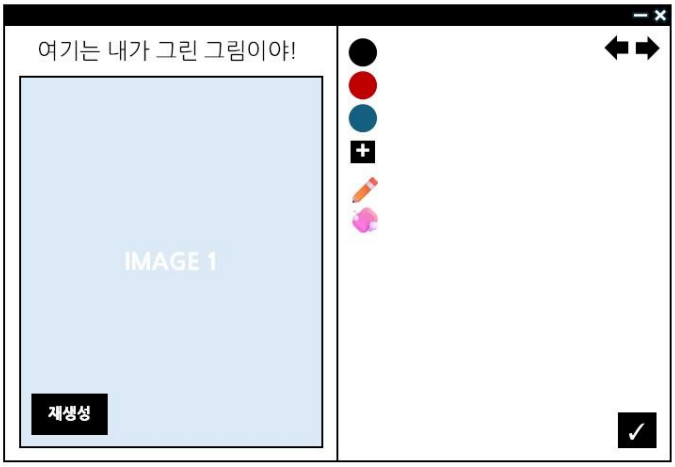

〈기능〉
- '사용 시작' 키를 누르면 기본 페이지 (1)로 이동

# 기본 페이지 (1)




〈기능〉
OpenAI API 를 활용하여
입력란에 받은 소재로 이미지 생성

# 기본 페이지 (2)

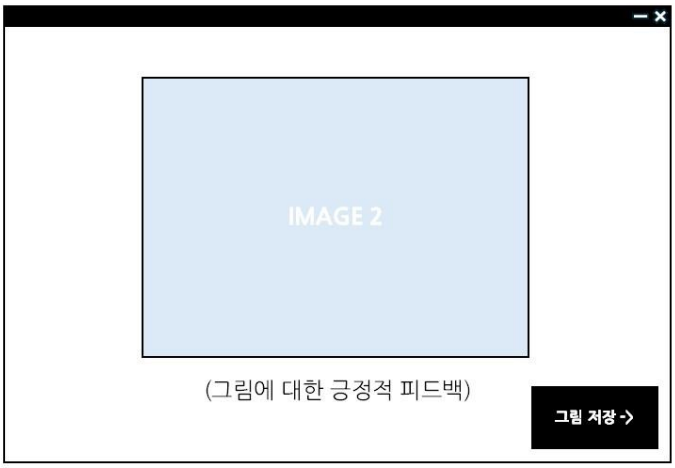



<기능>
- 창의 왼쪽: 기본 페이지 (2)에서 생성된 이미지 배치
  - '재생성' 키: 그림이 따라 그리기 애매할 경우 그림을 재생성 (OpenAI API)
- 창의 오른쪽: 그림판 기능
  - 기본 색상 선택란
  - '+' 키: 추가 색상을 선택할 수 있음 (새로운 색상 창 생성)
  - '연필' 키: 기본 상태. 그림판에 그릴 수 있음
  - '지우개' 키: 그림판에 그린 그림을 지움
  - '뒤로 가기', '앞으로 가기' 키
  - '완료' 키: 사용자가 그린 그림을 이미지의 형태로 저장한 후 그림 그리기를 종료

* 색상 창 예시

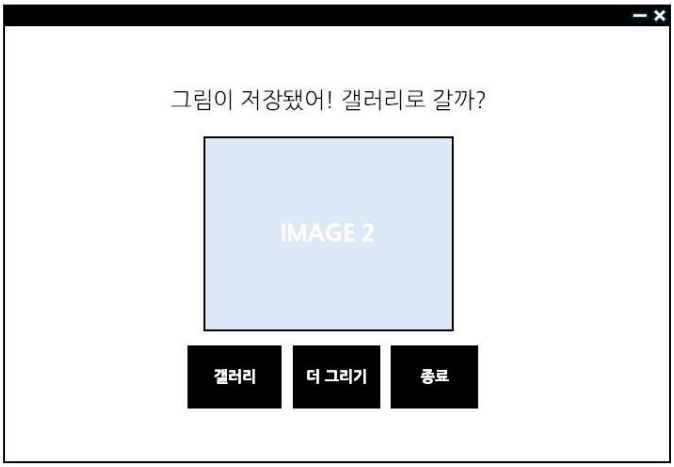


# 기본 페이지 (3)



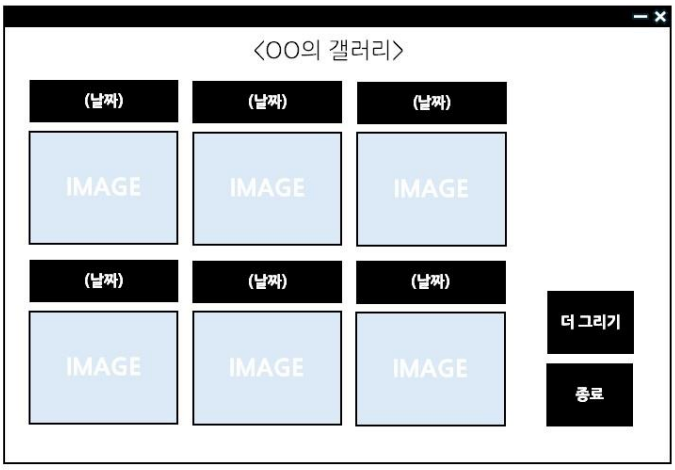

<기능>
- 기본 페이지 (2) 에서 그린 그림 출력
- Open AI API 로 그림에 대한 긍정적 피드백 출력
- '그림 저장' 키: 사용자의 '갤러리' 에 그림 저장 후 기본 페이지 (4)로 전환

# 기본 페이지 (4)




<기능>
- 기본 페이지 (2) 에서 그린 그림 출력
- '갤러리' 키: 갤러리 페이지로 전환
- '더 그리기' 키: 기본 페이지 (1)로 전환
- '종료' 키: 프로그램 종료

# 갤러리 페이지




<기능>
- 기존 그림 날짜와 함께 출력
- 그림 클릭 시 확대 창 출력
- '더 그리기' 키: 기본 페이지 (1)로 전환
- '종료' 키: 프로그램 종료

✓ 의사 코드

채팅

```
# 첫 방문 페이지(1)
print "안녕! 이름이 뭐야?"
name <- input()
create_button("Enter")

if button_clicked("Enter") then
    goto 첫 방문 페이지(2)

# 첫 방문 페이지(2)
print "반가워, " + name + "! 나는 XX라고 해. 우리 같이 그림을 그려보자!"
create_button("사용 방법")
create_button("사용 시작")

if button_clicked("사용 방법") then
    goto 첫 방문 페이지(3)
else if button_clicked("사용 시작") then
    goto 기본 페이지(1)

# 첫 방문 페이지(3)
print "사용 방법"
print “(간단한 사용 설명)”
create_button("사용 시작")

if button_clicked("사용 시작") then
    goto 기본 페이지(1)

# 기본 페이지(1)
print "오늘은 무슨 그림을 그릴까?"
subject <- input()
image <- call_openai_api(subject)

# 기본 페이지(2)
print "여기는 내가 그린 그림이야!"
display(image)
create_button("재생성")

if button_clicked("재생성") then
    image <- call_openai_api(subject)
    display(image)

# 기본 페이지(3)
user_drawing <- input( )
display(user_drawing)
feedback <- call_openai_feedback_api(user_drawing)
print feedback

create_button("그림 저장")
```

| | |
|---|---|
| | if button_clicked("그림 저장") then<br>  save_to_gallery(user_drawing)<br>  goto 기본 페이지(4)<br><br># 기본 페이지(4)<br>print "그림이 저장됐어! 갤러리로 갈까?"<br>display(user_drawing)<br>create_button("갤러리")<br>create_button("더 그리기")<br>create_button("종료")<br><br>if button_clicked("갤러리") then<br>  goto gallery_page()<br>else if button_clicked("더 그리기") then<br>  goto 기본 페이지(1)<br>else if button_clicked("종료") then<br>  exit_program() |
| 그림판 | 1. 그림판 생성<br>2. 그리기<br>3. 색상 선택<br>4. 펜 크기 선택<br>5. 그림 그리기 종료<br><br>1. #create canvas<br>Canvas = Canvas(window, width=500, height=500)<br><br>2. #paint<br>Def Paint:<br>  (x1, y1) = (x+1, y+1)<br>  (x2, y2) = (x-1, y-1)<br>  Canvas. Create_oval(x1, y1, x2, y2, , fill=pen_color, outline=pen_color,<br>  width=pen_size)<br><br>3. #color<br>Pen_color = black<br>Def blue:<br>  Global pen_color<br>  Pen_color = “blue”<br>Button_blue=Button(window, width=5, bg=blue, command=blue)<br>Button_blue. place at top<br><br>#change “blue” into “other color”, wirte other color code as well<br><br>4. #size<br>Pen_size = 5<br>Def size_up:<br>  Global pen_size<br>  pen_size += 1 |

Def size_down:
Global pen_size
If pen_size > 1
Pen_size -= 1

Button_size_up = button(window, text='굵게', width=5, command=size_up)
Button_size_up = button(window, text='얇게', width=5, command=size_down)
Button_size_up and Button_size_down place at buttom

5. #end
Def end_canvas:
Download Canvas into image_storage
Break
Button_end = button(window, text='끝', width=5, command=end_canvas)

| 갤러리 | 자연어 | 1. 사진을 저장할 첫 번째 디렉토리 생성 또는 확인<br>- "image_storage"라는 기본 디렉토리 생성<br>- 해당 디렉토리가 없으면 생성<br><br>2. 현재 저장된 사진의 개수 확인<br>- 첫 번째 디렉토리에서 사진 파일의 개수를 셈<br>- 만약 사진이 16개 이상이면 새로운 디렉토리를 생성<br>예: "image_storage_2", "image_storage_3" 등과 같은 이름으로 디렉토리 생성<br><br>3. 사용자로부터 사진 파일을 받는 기능 구현<br>- 파일 탐색기(파일 선택 창)를 통해 사용자가 사진을 선택하도록 함<br>- 선택한 사진을 현재 디렉토리에 저장<br>- 저장 후, 저장된 파일 수를 확인하여 16개가 넘으면 다음 디렉토리로 이동<br><br>4. 사진 갤러리(4x4 그리드) UI 초기화<br>- UI 창 생성<br>- 그리드를 4x4으로 설정 (16개의 셀)<br>- 셀 하나당 사진 하나를 표시할 영역 확보<br><br>5. 저장된 모든 디렉토리에서 사진 불러오기<br>- "image_storage", "image_storage_2", "image_storage_3" 등 모든 디렉토리에서 사진 파일 목록을 가져옴<br><br>6. 사진을 그리드에 표시<br>- 불러온 사진 파일들을 차례대로 그리드에 삽입<br>- 사진이 16개 미만이면 빈 칸으로 남김<br>-현재 날짜와 시간 생성<br>-날짜를 원하는 형식으로 변환<br>-오늘의 날짜 출력<br><br>7. 그리드 및 UI 업데이트<br>- 사진이 추가되거나 삭제될 때마다 그리드 업데이트<br>- 사진을 다시 로드하고 그리드를 다시 그린다<br><br>8. 프로그램 종료 시 UI 및 리소스 정리 |
|---|---|---|

| | | |
|---|---|---|
| | 의사 코드 | START<br><br>1. FUNCTION EnsureDirectoryExists(directory)<br>IF NOT EXISTS(directory) THEN<br>CREATE directory<br>END IF<br>END FUNCTION<br><br>2. FUNCTION GetNextDirectory() RETURNS directory<br>SET i = 1<br>WHILE TRUE DO<br>IF i > 1 THEN<br>SET current_directory = "image_storage_" + i<br>ELSE<br>SET current_directory = "image_storage"<br>END IF<br>CALL EnsureDirectoryExists(current_directory)<br>SET num_images = COUNT_FILES(current_directory)<br><br>IF num_images < 16 THEN<br>RETURN current_directory<br>END IF<br>SET i = i + 1<br>END WHILE<br>END FUNCTION<br><br>3. FUNCTION SaveImage(file_path)<br>SET save_directory = CALL GetNextDirectory()<br>SET image_name = GET_NAME(file_path)<br>SET destination = save_directory + "/" + image_name<br>COPY file_path TO destination<br>CALL LoadImages()<br>END FUNCTION<br><br>4. FUNCTION SelectImage()<br>SET file_path = OPEN_FILE_DIALOG("Select an image file")<br>IF file_path IS NOT NULL THEN<br>CALL SaveImage(file_path)<br>END IF<br>END FUNCTION<br><br>5. FUNCTION LoadImages()<br>SET all_images = []<br>SET i = 1<br>WHILE TRUE DO<br>IF i > 1 THEN<br>SET current_dir = "image_storage_" + i<br>ELSE<br>SET current_dir = "image_storage"<br>END IF<br>IF NOT EXISTS(current_dir) THEN<br>BREAK |

```
6. FUNCTION DisplayImages(images)
   FOR index FROM 0 TO 15 DO
      IF index < LENGTH(images) THEN
         SET img_path = images[index]
         DISPLAY_IMAGE(img_path, index)
      ELSE
         DISPLAY_EMPTY_CELL(index)
      END IF
   END FOR
END FUNCTION

7. FUNCTION Main()
   INITIALIZE GUI WINDOW
   CREATE BUTTON "Select Image" WITH ACTION SelectImage
   CALL LoadImages()
   START GUI EVENT LOOP
END FUNCTION

CALL Main()

END
```

✓ 비고

1. 프로그램이 딱딱한 목록 형식보다 대화 형식을 띄는 등 친숙한 사용자 경험을 제공하고자 함. (OpenAI API 활용)
2. 치료 목적이라고 명시하지는 않으나, 프로그램의 독창성 확보를 위해 심리 치료 개념을 차용하고자 함 (그림판 중 지우개 기능을 제외하는 등 (무조건적 수용))
3. 이후 활동에서 채팅 기능을 더욱 다양하게 활용할 방안을 고안하고자 함.

✓ 맡은 역할

| | |
|---|---|
| 유서현 | 의사 코드 작성 (채팅 기능) |
| 김채현 | 의사 코드 작성 (그림판 기능) |
| 도건우 | 내용 구조, 알고리즘 작성 |
| 양정현 | 구현 방안 고안, 사용자 인터페이스 고안, 취합 후 보고서 작성 |
| 유서진 | 의사 코드 작성 (갤러리 기능) |

✓ 앞으로의 역할

각자의 분야를 상세히 공부하고 분야 별로 해당 분야를 맡은 조원이 돌아가며 주도적으로 프로그램을 구현할 예정임.

| | |
|---|---|
| 유서현 | API 활용 |
| 김채현 | Figma를 이용한 디자인 |
| 도건우 | Tkinter를 이용한 그림판 구현 |
| 양정현 | API 활용, Figma를 이용한 디자인 |
| 유서진 | Tkinter를 이용한 전반적인 프로그램 구현 |

# <중간보고서_02_2_29아나>

**-프로그램 구현 코드**

from tkinter import *

from tkinter import colorchooser #색상 선택의 다양성을 추가하기 위해서 모듈 추가

from PIL import Image, ImageTk, ImageGrab # PIL(Pillow)의 이미지 처리 관련 클래스 불러옴

import json

import os # 운영체제와의 상호작용을 위한 OS 모듈을 불러옴

from datetime import datetime

import openai # OpenAI의 API를 사용하기 위해 openai 라이브러리를 불러오는 코드. OpenAI의 DALL-E 모델을 사용해 이미지를 생성하고, ChatGPT 모델을 사용해 긍정적인 피드백을 생성

from io import BytesIO #OpenAI에서 생성된 이미지의 URL을 통해 이미지 데이터를 가져온 후, 이 데이터를 BytesIO 메모리에 저장. 저장된 데이터는Pillow의 Image.open 메서드를 통해 실제 이미지로 변환되어 GUI에서 보여짐.

import requests #OpenAI API는 이미지를 URL 형태로 반환하므로, 해당 URL에서 이미지를 다운로드하려면 requests모듈을 사용하여 데이터를 가져와야 함.

from tkinter import messagebox # Tkinter의 메시지 박스 기능 불러옴

# OpenAI API 키 설정(API 호출을 위해서는 OpenAI 계정의 API 키가 필요)

openai.api_key = "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"

win = Tk()

```
win.geometry("1000x500")
win.title("그림 프로그램")
win.option_add("*Font","나눔고딕OTF 13")
win.resizable(False, False)

data_file = 'data.json'

def save_data(data):
    with open(data_file, 'w') as f:
        json.dump(data, f)

def load_data():
    if os.path.exists(data_file):
        with open(data_file, 'r') as f:
            return json.load(f)
    return {}

def paint(event):
    x1, y1 = event.x-1, event.y-1
    x2, y2 = event.x+1, event.y+1
    canvas.create_oval(x1, y1, x2, y2, fill=pen_color, outline = pen_color,  width = pen_size) # 마우스를 사용하여 그림을 그리는 부분은 paint()함수에서 처리됨. 이함수는 canvas위젯에 마우스를 드래그할 때마다 호출되도록 코딩. # #그림판 생성부분에서 <B1-Motion>이벤트는 마우스 왼쪽 버튼을 누른 상태로 마우스를 움직일 때 발생하도록 코딩. 그때마다 위의 paint함수가 호출되어 작은 원을 그리며 선을 그리도록 코딩. 이때 선의 굵기는 앞서 설명한 pen_size에의해 결정되고, 색은 pen_color로 결정되도록 코딩.

#펜 색깔 변경 부분
```

```
def change_color(color):
    global pen_color
    pen_color = color
#각 색깔을 누를 때마다 btn_색깔 형식의 함수 호출하도록 명령
def choose_color():  #그림판처럼 색상 선택 모듈 소환
    global pen_color
    color_code = colorchooser.askcolor(title="색상 선택")
    if color_code[1]:  # 색상이 선택되었는지 확인
        pen_color = color_code[1]


def size_up():
    global pen_size
    pen_size += 2

def size_down():
    global pen_size
    if pen_size>2:
        pen_size -= 2

data = load_data()

def frame1():
    global frame1_name
    frame1 = Frame(win, width=1000, height=500)
```

```
    frame1.place(x=0, y=0)

    frame1_text = Label(frame1, text = "안녕! 이름이 뭐야?")
    frame1_text.place(relx=0.5, rely=0.4, anchor='center')

    frame1_name = Entry(frame1)
    frame1_name.place(relx=0.5, rely=0.45, anchor='center')

    data['name'] = frame1_name.get()
    data['first_run'] = False

    save_data(data)

    frame1_btn = Button(frame1, text = "입력", fg = "white", bg = "black", command = lambda: save())
    frame1_btn.place(relx=0.5, rely=0.525, anchor='center')
    #이 창에서 받은 정보 저장해야함
    #이 창은 한번 등장해야함

def save():
    data['name'] = frame1_name.get()
    data['first_run'] = False
    save_data(data)
    os.makedirs(f"user_images/{data['name']}", exist_ok=True)
    frame2()
```

```
def frame2():
    global user_name
    for widget in win.winfo_children():
        widget.destroy()

    data = load_data()

    frame2 = Frame(win, width=1000, height=500)
    frame2.place(x=0, y=0)

    user_name = data.get('name')
    frame2_text = Label(frame2, text = "반가워 %s!!₩n 나는 노스체라고 해.₩n 우리 같이 그림을 그려보자!"%user_name)
    frame2_text.place(relx=0.5, rely=0.4, anchor='center')

    frame2_btn1 = Button(frame2, text = "사용 방법",fg = "white", bg = "black",
                         width = 10, command = lambda: frame3())
    frame2_btn1.place(relx=0.43, rely=0.55, anchor='center')

    frame2_btn2 = Button(frame2, text = "사용 시작",fg = "white", bg = "black",
                         width = 10, command = lambda: frame4())
    frame2_btn2.place(relx=0.56, rely=0.55, anchor='center')
    #로봇 이름 생각해보자
```

```
def frame2_2():

    global user_name
    data = load_data()

    frame2_2 = Frame(win, width=1000, height=500)
    frame2_2.place(x=0, y=0)

    user_name = data.get('name')
    frame2_2_text = Label(frame2_2, text = "안녕 %s!!₩n오늘도 같이 그림을 그려보자!"%user_name)
    frame2_2_text.place(relx=0.5, rely=0.4, anchor='center')

    frame2_2_btn = Button(frame2_2, text = "사용 시작",fg = "white", bg = "black",
                          width = 10, command = lambda: frame4())
    frame2_2_btn.place(relx=0.5, rely=0.55, anchor='center')
    #로봇 이름 생각해보자

def frame3():
    for widget in win.winfo_children():
        widget.destroy()

    frame3 = Frame(win, width=1000, height=500)
    frame3.place(x=0, y=0)
```

```
    frame3_text = Label(frame3, text = "사용 방법₩n₩n1.그리고 싶은 주제를 입력한
다!₩n₩
    2.노스체가 그려준 그림을 참고해서 나만의 그림을 그려보자!₩n3.그림을 저장하고 갤
러리를 확인해보자!")
    frame3_text.place(relx=0.5, rely=0.3, anchor='center')

    frame3_btn = Button(frame3, text = "사용 시작",fg = "white", bg = "black",
                        width = 10, command = lambda: frame4())
    frame3_btn.place(relx=0.5, rely=0.55, anchor='center')

def frame4():
    global frame4_topic, image_title
    for widget in win.winfo_children():
        widget.destroy()

    frame4 = Frame(win, width=1000, height=500)
    frame4.place(x=0, y=0)

    frame4_text = Label(frame4, text="오늘은 무슨 그림을 그릴까?")
    frame4_text.place(relx=0.5, rely=0.4, anchor='center')

    frame4_topic = Entry(frame4)
    frame4_topic.place(relx=0.5, rely=0.45, anchor='center')

    def save_title():
        global image_title
```

```
        image_title = frame4_topic.get()  # 제목 저장
        frame5()  # 다음 단계로 이동

    frame4_btn = Button(frame4, text="입력", fg="white", bg="black",
command=save_title)
    frame4_btn.place(relx=0.5, rely=0.525, anchor='center')

    # 입력 받은 주제로 API 사용하여 출력해야! -> 함수로 정의해볼 것

# DALL-E 모델을 통해 이미지를 생성하는 함수
def generate_image_from_topic(topic):
    try:
        response = openai.Image.create(prompt=topic, n=1, size="512x512")  # 최신 인터
페이스 사용
        image_url = response['data'][0]['url']

        # 이미지 다운로드
        img_response = requests.get(image_url)
        img_data = BytesIO(img_response.content)
        return Image.open(img_data)

    except Exception as e:
        messagebox.showerror("이미지 생성 오류", f"이미지 생성 중 오류가 발생했습니다:
{e}")
        return None
```

```
def regenerate_image():
    # Generate a new image from the previously entered topic
    global img_label_0
    generated_image = generate_image_from_topic(image_title)

    if generated_image:
        # Update the label with the new image
        img_tk = ImageTk.PhotoImage(generated_image)
        img_label_0.config(image=img_tk)
        img_label_0.image = img_tk  # Keep a reference to avoid garbage collection
    else:
        messagebox.showerror("이미지 생성 오류", "이미지를 생성할 수 없습니다.")

#그림판 생성 부분 frame 5안에 넣어 코딩
def frame5():
    global canvas, pen_color, pen_size, img_label_0
    for widget in win.winfo_children():
        widget.destroy()

    frame5_1 = Frame(win, width=500, height=500, bg = "lightblue")
    frame5_1.pack(side=LEFT) #그려진 그림을 왼쪽에 표시

    frame5_2 = Frame(win, width=500, height=500,
    highlightbackground = "black", highlightthickness = 1)
    frame5_2.pack(side=RIGHT) # 그림을 그릴수 있는 캔버스를 오른쪽에 표시
```

```
    frame5_1_text = Label(frame5_1, text = "여기는 내가 그린 그림이야")
    frame5_1_text.place(relx=0.5, rely=0.1, anchor='center')

    generated_image = generate_image_from_topic(image_title)

    if generated_image:
        img_tk_0 = ImageTk.PhotoImage(generated_image)
        img_label_0 = Label(frame5_1, image=img_tk_0)
        img_label_0.image = img_tk_0
        img_label_0.place(relx=0.5, rely=0.5, anchor='center')
    else:
        Label(frame5_1, text="이미지를 생성할 수 없습니다.").place(relx=0.5, rely=0.5,
anchor='center')

    # "재생성" 버튼의 command를 regenerate_image 함수로 수정
    frame5_1_btn = Button(frame5_1, text="재생성", width=5, fg="white", bg="black",
command=regenerate_image)
    frame5_1_btn.place(relx=0.1, rely=0.9, anchor='center')

    canvas = Canvas(frame5_2,width=400, height=500)
    canvas.pack(side = RIGHT)
    canvas.bind("<B1-Motion>", paint) #마우스의 왼쪽 부분을 눌렀을 때 그림을 그릴 수
있도록 코딩함

    option_frame = Frame(frame5_2, width = 60, height = 500, bg = 'gray')
```

```
    option_frame.pack(side=LEFT)


    pen_color = "black"
    pen_size = 6


    btn_black = Button(option_frame, width = 5, bg = "black", command = lambda: change_color("black"))
    btn_black.place(x=0, y=0)


    btn_red = Button(option_frame, width = 5, bg = "red", command = lambda: change_color("red"))
    btn_red.place(x=0, y=40)


    btn_blue = Button(option_frame, width = 5, bg = "blue", command = lambda: change_color("blue"))
    btn_blue.place(x=0, y=80)
    btn_colors = Button(option_frame, width = 5, text = "다른색", font = ("나눔고딕 OTF",'12'),command = choose_color)
    btn_colors.place(x=0, y=120)


    btn_size_up = Button(option_frame, text = "굵게", width = 5, font = ('나눔고딕 OTF','12'), command = size_up)
    btn_size_up.place(x=0, y = 160)


    btn_size_down = Button(option_frame, text = "가늘게", width = 5, font = ('나눔고딕 OTF','12'), command = size_down)
    btn_size_down.place(x=0, y = 200)
```

```
    #btn_erase = Button(frame5_2, text = "지우기", width = 5, font = ('나눔고딕OTF','12'), command = lambda: canvas.delete("all"))

    #btn_erase.place(x=0, y = 200)


    frame5_2_btn = Button(option_frame, text = "✓", fg = "white", bg = "black", command=lambda: [save_image(), frame6()])

    frame5_2_btn.place(relx=0.5, rely=0.9, anchor='center')


    #여러 색 고를 수 있는 미니 창을 만들어야 함!!

    #생성 함수 만들어서 재생성 버튼에 사용해야

    #뚝뚝 끊기는 선 수정해야

    #한번 뒤로가기 기능 불가능한가 ㅜㅜ

    # 그림판의 그림 저장해야


def save_image():

    # 전역 변수로 사용되는 변수들 선언

    global current_time, user_image, image_count, img_path, user_folder, image_title


    # 사용자 이미지 폴더 경로 설정 및 폴더가 없으면 생성

    user_folder = f"user_images/{user_name}"

    os.makedirs(user_folder, exist_ok=True)


    # 윈도우의 좌상단 x, y 좌표 및 너비와 높이를 가져와 스크린샷 영역 설정

    x = win.winfo_rootx()  # 윈도우 좌상단 x좌표

    y = win.winfo_rooty()  # 윈도우 좌상단 y좌표
```

```
    width = win.winfo_width()  # 윈도우 너비
    height = win.winfo_height()  # 윈도우 높이

    # 캡처 영역(bbox)을 지정하여 스크린샷 저장
    user_image = ImageGrab.grab(bbox=(x + 30, y, x + width + 250, y + height + 100))

    # 현재 시간을 특정 포맷으로 가져와 파일명 등에 사용할 준비
    current_time = datetime.now().strftime("%Y%m%d_%H%M%S")  # 예: 20241110_153045
    formatted_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")  # 예: 2024-11-10 15:30:45

    # 사용자 폴더 내 이미지 수를 기반으로 번호 지정
    image_count = len(os.listdir(user_folder)) + 1

    # 이미지 파일 경로 설정 - 제목과 날짜 포함하여 파일명 생성
    img_path = f"{user_folder}/{image_count}_{image_title}_{current_time}.png"
    user_image.save(img_path)  # 이미지 저장

#이름에 무슨 그림인지도 추가해야...

# 긍정적인 피드백을 생성하는 함수
def generate_positive_feedback():
    try:
        prompt_text = "이 작품에 대해 긍정적이고 격려하는 코멘트를 해주세요. 미술 멘토처럼 작성해주세요."
```

```
        response = openai.ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[
                {"role": "system", "content": "You are an art mentor providing positive feedback in Korean."},
                {"role": "user", "content": prompt_text}
            ],
            max_tokens=200,  # 더 긴 피드백을 위해 max_tokens 값을 증가
            temperature=0.7
        )
        feedback_text = response.choices[0].message['content'].strip()
        return feedback_text
    except Exception as e:
        messagebox.showerror("피드백 생성 오류", f"피드백 생성 중 오류가 발생했습니다: {e}")
        return "피드백을 생성할 수 없습니다."


def frame6():
    global user_image
    for widget in win.winfo_children():
        widget.destroy()

    frame6 = Frame(win, width=1000, height=500)
    frame6.place(x=0, y=0)
```

```
    user_image = user_image.resize((300, 300))
    user_image_tk = ImageTk.PhotoImage(user_image)

    user_image_print = Label(frame6, image=user_image_tk)
    user_image_print.place(relx=0.5, rely=0.3, anchor='center')
    user_image_print.image = user_image_tk

    frame6_btn1 = Button(frame6, text="갤러리", fg="white", bg="black",
                         width=10, command=lambda: gallery_frame())
    frame6_btn1.place(relx=0.38, rely=0.7, anchor='center')

    frame6_btn2 = Button(frame6, text="더 그리기", fg="white", bg="black",
                         width=10, command=lambda: frame4())
    frame6_btn2.place(relx=0.5, rely=0.7, anchor='center')

    frame6_btn3 = Button(frame6, text="종료", fg="white", bg="black",
                         width=10, command=lambda: win.destroy())
    frame6_btn3.place(relx=0.62, rely=0.7, anchor='center')

    # 긍정적인 피드백 생성 및 표시
    feedback_text = generate_positive_feedback()
    feedback_label = Label(frame6, text=feedback_text, wraplength=600, justify="center",
font=("나눔고딕OTF", 12))
    feedback_label.place(relx=0.5, rely=0.85, anchor="center")
```

```
def gallery_frame():
    # 기존 위젯 제거
    for widget in win.winfo_children():
        widget.destroy()

    # 스크롤 가능하도록 Canvas와 Scrollbar 생성
    canvas2 = Canvas(win)
    scrollbar = Scrollbar(win, orient="vertical", command=canvas2.yview)
    scrollable_frame = Frame(canvas2)

    # 스크롤 가능한 프레임을 Canvas에 추가
    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas2.configure(scrollregion=canvas2.bbox("all"))
    )

    canvas2.create_window((0, 0), window=scrollable_frame, anchor="nw")

    # 스크롤바 배치
    scrollbar.pack(side="right", fill="y")
    canvas2.pack(side="left", fill="both", expand=True)
    canvas2.config(yscrollcommand=scrollbar.set)

    # 이미지 그리드 생성
    create_image_grid(scrollable_frame)
```

```
    # 버튼을 이미지 그리드 오른쪽에 배치
    gframe_btn1 = Button(scrollable_frame, text="더 그리기", fg="white", bg="black", width=10, command=lambda: frame4())
    gframe_btn1.grid(row=0, column=3, padx=5, pady=5)  # 첫 번째 행의 마지막 열 오른쪽에 배치

    gframe_btn2 = Button(scrollable_frame, text="종료", fg="white", bg="black", width=10, command=lambda: win.destroy())
    gframe_btn2.grid(row=1, column=3, padx=5, pady=5)  # 두 번째 행의 마지막 열 오른쪽에 배치

#이미지 옆에 날짜, 무슨 그림인지 떴으면

def load_images():
    images = []
    # 사용자 폴더에서 이미지 파일 목록 가져오기
    for filename in os.listdir(user_folder):
        if filename.endswith(('.png', '.jpg', '.jpeg', '.gif')):  # 이미지 확장자 필터링
            img_path = os.path.join(user_folder, filename)
            img = Image.open(img_path)
            img = img.resize((200, 250))  # 원하는 크기로 조정
            img_tk = ImageTk.PhotoImage(img)  # Tkinter에서 사용할 수 있도록 변환
            # Extract the image title from the filename (the part before the first '_')
            image_title = filename.split('_')[1]  # Assuming the title is stored after the first '_'
            images.append((img_path, img_tk, image_title))  # Include title in the tuple
```

```
    return images

def create_image_grid(frame):
    images = load_images()  # 이미지 로드
    rows = (len(images) + 2) // 3  # 이미지 개수에 따라 행 수 계산 (3개씩 배치)
    cols = 3  # 한 행에 3개

    for r in range(rows):
        for c in range(cols):
            index = r * cols + c
            if index < len(images):  # 이미지가 있는 경우에만 표시
                img_path, img_tk, image_title = images[index]  # 경로, img_tk, 제목을 튜플로 받음
                img_btn = Button(frame, image=img_tk, command=lambda img_path=img_path, image_title=image_title: show_image(img_path, image_title))
                img_btn.image = img_tk  # Button에서 ImageTk 객체에 대한 참조 유지
                img_btn.grid(row=r, column=c, padx=5, pady=5)

def show_image(img_path, image_title):
    top = Toplevel()
    top.title("이미지 보기")

        try:
        # 파일 이름에서 날짜와 제목 부분 추출
        img_name = os.path.basename(img_path)  # 이미지 파일의 이름만 추출
        parts = img_name.split('_')  # 파일명 구분자로 나눔
```

```
        if len(parts) >= 3:
            title = parts[1]  # 제목 부분 추출
            date_str = parts[2]  # 날짜 부분 추출
            formatted_date = f"{date_str[:4]}-{date_str[4:6]}-{date_str[6:8]}"  # 'YYYY-MM-DD' 형식으로 날짜 생성
            time_str = f"{date_str[9:11]}:{date_str[11:13]}:{date_str[13:]}"  # 'HH:MM:SS' 형식으로 시간 생성
        else:
            # 파일명에서 제목과 날짜를 추출할 수 없는 경우 기본값 설정
            title = "제목 없음"
            formatted_date = "날짜 없음"
            time_str = "시간 없음"

        # 이미지 로드 및 크기 조정
        img = Image.open(img_path)  # 이미지 열기
        img = img.resize((400, 500), Image.LANCZOS)  # 크기 조정
        img_tk_resized = ImageTk.PhotoImage(img)  # Tkinter에서 사용할 수 있도록 이미지 변환

        # 이미지 라벨 생성 및 추가
        img_label = Label(top, image=img_tk_resized)
        img_label.image = img_tk_resized  # 이미지 참조 유지 (GC로부터 보호)
        img_label.pack()

        # 제목, 날짜, 시간 라벨 추가
```

```
            title_label = Label(top, text=f"제목: {title}", font=("나눔고딕OTF", 12, "bold"))
            title_label.pack()
            date_label = Label(top, text=f"날짜: {formatted_date}")
            date_label.pack()
            time_label = Label(top, text=f"시간: {time_str}")
            time_label.pack()

        except IndexError:
            # 파일명에서 필요한 정보를 찾을 수 없는 경우 오류 메시지 표시
            error_label = Label(top, text="정보를 찾을 수 없습니다.", fg="red")
            error_label.pack()

        # 닫기 버튼 추가
        close_button = Button(top, text="닫기", command=top.destroy)
        close_button.pack(pady=10)

        top.geometry("400x600")

if data.get('first_run', True):
    frame1()
else:
    frame2_2()
win.mainloop()
```

**-함수 설명**

generate_image_from_topic(topic)

이 함수는 DALL-E API를 사용해 주어진 주제(topic)에 맞는 이미지를 생성합니다.

1. openai.Image.create: OpenAI의 DALL-E 모델을 호출하여 주어진 prompt(주제)에 맞는 이미지를 생성합니다. n=1은 이미지 한 개를 생성하겠다는 뜻이고, size="512x512"는 이미지의 크기를 설정합니다.
2. image_url = response['data'][0]['url']: OpenAI API는 이미지 URL을 반환하며, 이를 통해 이미지를 다운로드합니다.
3. requests.get(image_url): URL에서 이미지 데이터를 가져옵니다. 요청이 성공하면 이미지 데이터가 img_response.content에 저장됩니다.
4. BytesIO(img_response.content): 가져온 이미지 데이터를 BytesIO 객체에 저장해 메모리에서 처리합니다.
5. Image.open(img_data): BytesIO 데이터를 Pillow의 Image.open을 사용해 Pillow 이미지 객체로 변환하여 반환합니다.

regenerate_image()

이 함수는 generate_image_from_topic 함수를 호출해 이미지를 다시 생성하여 업데이트합니다.

1. generated_image = generate_image_from_topic(image_title): image_title을 이용해 새로운 이미지를 생성합니다.
2. img_label_0.config(image=img_tk): Label 위젯의 이미지를 새로운 이미지로 업데이트합니다.
3. img_label_0.image = img_tk: Tkinter에서 이미지 객체를 유지하기 위해 변수를 할당하여 가비지 컬렉션을 방지합니다.

generate_positive_feedback()

이 함수는 OpenAI의 gpt-3.5-turbo 모델을 사용해 사용자 그림에 대한 긍정적인 피드백을 생성합니다.

1. prompt_text: 피드백의 내용을 지정하기 위한 텍스트로, "이 작품에 대해 긍정적이고 격려하는 코멘트를 해주세요."로 설정합니다.
2. openai.ChatCompletion.create: ChatGPT 모델을 호출하여 피드백을 생성합니다.

- model="gpt-3.5-turbo"는 사용할 모델을 지정합니다.
- messages: 대화 내용으로, 첫 번째 메시지에 "You are an art mentor providing positive feedback in Korean."을 지정하여 한국어로 긍정적인 피드백을 작성하도록 설정합니다.
- max_tokens=200: 응답 길이를 설정합니다.
- temperature=0.7: 응답의 창의성 정도를 설정합니다.

**-추가 설명**

<프로그램 골격>

- 이미지가 중요한 프로그램인 만큼 창 크기를 변경 불가하게 설정함

- 폰트는 나눔고딕 13으로 설정함

- 프레임을 통해 화면 전환.

- 프레임 전환: 프레임 함수(기존 프레임을 삭제하고 새로운 위젯을 배치)를 생성한 후 첫 실행 시 첫 프레임 함수로 프레임을 부르고, 그 이후는 버튼을 통해 다음 프레임 함수를 호출함.

<프레임 위젯>

- 버튼: bg, fg 등의 파라미터를 통해 검정 버튼 하얀 글씨로 형식 통일하고자 함/ 버튼이 수행하는 명령이 복잡한 경우에는 함수를 따로 만들어 삽입, 간단한 경우 lambda 함수 이용

- 위젯 위치 설정: relx,rely 등을 이용하여 설정/anchor로 정렬/ 버튼이 많은 경우 특정 좌표를 지정하여 위치 설정

<이름 저장>

- data_json 파일 생성

- 저장 함수(save_data), 로딩 함수(load_data) 만들기

- frame1에서 받은 이름을 save_data()를 통해 파일에 저장

- 이후 프레임에서 load_data()로 데이터를 로딩, 이름으로 사용

<실행 회차에 따른 차이>

첫 회차에 frame1이 호출되어 이름을 입력 받고, 그 창은 다시 호출되지 말아야 함.

- frame1()을 실행할 시 data_json 파일에서 first_run 항목이 false로 변경됨

- 프레임 호출 시 first_run이 True면 frame1을, False면 frame2_2를 호출함

<그림판 구현>

- 프레임을 세개의 하위 프레임으로 세분

- 왼쪽부터 생성된 이미지가 있는 프레임/ 옵션 프레임/ 캔버스 프레임

- 캔버스 프레임에서 paint함수를 통해 마우스 클릭 시 점이 생성되도록 구현, 드래그 시 선을 그릴 수 있음.

- paint함수는 pen_color, pen_size 등으로 조정되고 해당 조정은 옵션 창에서 가능함.

<API 구현 적용 중 오류 수정>

- 기본 프로그램에 이미지 생성 함수가 잘 구현되도록 보조

- img_tk 등의 변수가 캔버스 저장 시 등장하는 변수와 겹쳐 변수명 변경

- 정상적 작동을 위해 open ai 모듈 다운그레이드 0.28

<그림판 이미지 저장 코드 수정>

- 그림을 캡쳐하여 저장하는 형태

- 기존 캔버스 프레임은 옵션창과 캔버스의 구분이 없었는데, 그림만 캡쳐하기 용이하게 하기 위해 창을 구분함.

- 창의 화면 상 좌표를 상대적으로 잡는 함수를 이용하여 좌표 설정하고자 함

<그림 저장 시 날짜 입력되도록 코드 추가>

- datetime 클래스의 메서드인 datetime.now()를 활용하여 저장하는 그림의 파일명이 날

짜를 포함하도록 함.

<갤러리 프레임 수정>

- 스크롤 가능하도록 캔버스와 스크롤바 추가
- 캔버스에 맞게 갤러리 프레임 수정

**-맡은 역할**

김채현:

- Rogers의 인간중심 상담원리와 선행 미술심리치료 사례를 검토. 이를 바탕으로 프로그램에 무조건적 긍정적 수용의 원리를 구현할 방안을 탐구함. '지우개 없는 그림판'을 통해 '실수를 지운다'는 자기 방어와 부정적인 내담자의 경험도 있는 그대로 받아들이고 이로부터 자유로울 수 있는 표현의 기회를 제공하기 위해 지우개 기능 삭제.
- 색상 선택의 다양성을 추가하기 위해서 '다른 색' 버튼에 색상 선택 화면 모듈 추가.

도건우:

- 그림판 생성관련 코드 공부 및 실현
- 그림판 생성 및 포인터 움직임 설명 추가

양정현:

- 그림 프로그램 창과 프레임 등 프로그램 골격 구현
- 프레임 내 위젯 구현 (frame1, 2, 2_2, 3, 4, 5_1, 5_2, 6, gallery_frame,)
- 실행 1회차에만 다른 화면이 뜨고 이름 저장할 수 있도록 구현
- 그림판 구현
- API 이미지 생성 함수 적용 중 코드 오류 수정
- 그림판 이미지 저장 코드 수정
- 그림 저장 시 날짜 입력되도록 코드 추가
- 갤러리 이미지 불러오기 코드 수정

유서진:

- 그림판 저장 버튼 구현
- 그림 저장 함수 정의
- 사용자 폴더 나열 함수 정의
- 사용자 폴더 그림 보여주는 함수 정의

- 이미지 크게 보여주는 함수 정의
- 윈도우 창 캡처 올바르게 되도록 수정
- frame4()에서 제목 저장하도록 수정
- 이미지 저장 함수에서 파일명에 제목과 날짜 추가
- 갤러리 이미지를 누르면 제목과 날짜 표시되도록 수정

유서현:

- open ai를 이용한 이미지 생성 기능
- open ai를 활용한 이미지 재생성 기능
- open ai를 활용한 피드백 생성 기능
- 이미지 재생성 기능이 가능하도록 재생성 버튼의 명령 기능 수정
- 보고서 작성

**-수정할 사항**

<open ai를 이용한 이미지 및 피드백 생성 관련 수정사항>

1. 이미지의 크기가 너무 커서 조금 더 작게 만들어야 함

2. 한국어로 주제를 입력할 시 주제와 다른 이미지가 나오는 것

3. 이미지 생성하는데 걸리는 시간이 매우 긴 점

4. 사용자가 그린 그림에 대한 피드백이 비슷한 점

5. pip install openai==0.28을 해야 이미지가 생성되는데 사용자가 이걸 하지 않아도 이미지가 생성되야함

6. API가 그림 느낌이 나는 이미지 생성해야함.지금은 사진에 가까움.

<프로그램의 기능 관련 수정사항>

1. 갤러리 이미지의 제목을 사용자의 니즈에 맞게 바꿀 수 있도록 수정
2. 최종 파이썬 프로그램을 앱의 형식으로 추출하기
3. 이름 입력 추후에 변경할 수 있는 항목을 마련해야 함.
4. 이름 입력 창에서 종료한 경우 다시 입력 창이 안 뜨는 문제 발생, 구현 순서 고민해봐야 함.
5. 빠른 드래그 시 점선으로 구현되는 것.
6. 이미지 캡쳐가 제대로 안됨.

# 최종보고서

<수정된 사항들>

**<open api 관련 코드 수정사항>**(유서현)

1. 이미지의 크기가 너무 커서 조금 더 작게 만들어야 함

-이미지 크기를 "512x512"에서 "256x256"으로 수정하여 "여기는 내가 그린 그림이야" 라는 문구가 보이도록 함.

2. 한국어로 주제를 입력할 시 주제와 다른 이미지가 나오는 것

- googletrans라이브러리 추가: Google Translate API를 통해 주제를 한국어에서 영어로 번
역.

- 번역 처리 추가: generate_image_from_topic()함수에서 Translator를 사용하여 입력 주제를
번역 후 DALL-E로 전달.

- 이미지 재생성에도 번역 반영: regenerate_image()함수에서도 번역된 주제를 사용.

3. 이미지 생성하는데 걸리는 시간이 매우 긴 점

- 비동기로 처리

generate_image_async는 이미지를 생성하는 작업을 별도의 스레드에서 수행함. Tkinter의 메인 이벤트 루프(UI 업데이트 등)는 차단되지 않고 계속 동작함.

이미지 생성이 완료되면 callback(image)으로 결과를 메인 스레드로 전달하여 UI를 업데이트함.

- 이미지 캐싱

image_cache딕셔너리를 사용하여 캐싱된 이미지를 저장하고 활용하고 있음. frame5()함

수에서 캐싱된 이미지를 먼저 확인함.

이를 통해 주제가 동일한 경우 캐싱된 이미지를 사용하여 불필요한 API 호출 및 이미지 다운로드를 방지함.

- 번역 캐싱

translation_cache딕셔너리를 사용하여 번역 결과를 저장하고 활용함.

generate_image_from_topic함수에서 번역 시 translate_text를 호출하여 캐싱된 번역 결과를우선적으로 확인함.동일한 주제를 반복적으로 번역하는 경우 캐싱된 번역을 반환하여 번역 API 호출을 방지

함.

4. 사용자가 그린 그림에 대한 피드백이 비슷한 점

-temperature을 0.7에서 1.0으로 늘려 피드백의 창의성을 증가시킴.

5. pip install openai==0.28을 해야 이미지가 생성되는데 사용자가 이걸 하지 않아도 이미지가 생성되야함

-라이브러리 설치 가이드 문서에 기재함.

6. API가 그림 느낌이 나는 이미지 생성해야함. 지금은 사진에 가까움.

- 스타일 드롭다운 추가: frame4에서 사용자가 스타일을 선택할 수 있도록 드롭다운 메뉴를 제공함.

- 스타일 전달: 선택된 스타일을 generate_image_from_topic 함수에 전달함.

- generate_image_from_topic 함수 수정: 스타일 매개변수를 받아서 프롬프트에 추가함.

- 색감 개선을 위한 추가 설정: 색감이 더 풍부하고 다양한 느낌을 주도록 프롬프트를 확장함.

예) "a watercolor painting of", "a colorful illustration of", "a digital artwork of", "a vibrant cartoon drawing of", "an abstract art piece of", "an oil painting of"

-위 프롬프트를 무작위 또는 선택적으로 사용하게 하여 색감과 스타일이 더 풍부해지게

함.

7. 사용자가 그린 그림에 데한 피드백이 길 경우 뒷부분이 잘려 보이지 않음

-token을 200에서 300으로 늘림

**<수정 사항>**(양정현)

1. 코드 순서대로 정리함
   (1) Import 모듈
   (2) 함수
      1) 데이터 저장
      2) 그림 판
      3) 갤러리
      4) API
      5) 설정 창
   (3) 프레임 함수
   (4) 실행 코드

   함수들도 더 종류가 세분화되는 경우 '#---------' 로 구분함
2. 선 도구 수정
   - 기존의 그림 판 그림 도구의 경우 마우스의 이동에 따라 점이 생성되는 방식을 이용하여, 일정 이상의 속도로 그렸을 때 점선 형태로 그려지는 문제 발생.
   - 그려지는 방식을 변경. 그림 시작 함수(마우스 클릭 시) -> 선 지속 -> 그림 종료 함수(마우스 클릭 종료 시)의 세 개 함수로 과정 세분화.
3. 이름 변경 설정 추가
   - 설정 창 추가: 설정 창 함수를 만들어 프레임 3, 4에 추가하여 첫 실행이 아니더라도 이름을 변경할 수 있도록 함.
   - 이후 변경된 이름이 프레임2_2와 프레임 3에 반영되도록 코드 수정

4. 첫번째 창에서 이름 입력 안 했을 때도 첫 화면 안 뜨는 거 구조 수정 (첫 화면에서 그냥 프로그램 종료하는 특이상황 등)
5. 피드백 말투 더 친근하고 짧게 수정 (API 프롬프트 수정)
6. 사용 설명 수정 (심리학 조사 자료 첨삭, 수정 후 프로그램에 반영)
7. 사용 설명 버튼 첫 실행 이후에도 확인 가능하도록 수정 (프레임 2_2에 사용 설명 버튼 추가)

**<캔버스 캡처 관련 코드 수정 사항>**(유서진)

**사용자의 기기에 따라 캡처되는 화면이 다름**

프로그래밍 초기에 캔버스를 캡처하기 위해 Pillow의 ImageGrab.grab()를 사용했음. Tkinter의 winfo_rootx()와 winfo_rooty() 함수는 위젯의 좌상단 좌표를 반환하지만 이는 논리적 좌표로 반환되는데, ImageGrab.grab()은 화면의 물리적 픽셀 좌표를 기준으로 캡처하기 때문에 좌표 계산이 정확하지 않으면 캡처 영역이 엉뚱한 위치를 포함하는 문제가 발생함.

이를 해결하기 위해 PyAutoGUI.screenshot()을 사용함.

PyAutoGUI.screenshot()도 물리적 픽셀 좌표를 기반으로 하긴 하지만 ImageGrab.grab()와 달리 DPI 비율이 자동 처리되고, 윈도우 창 테두리 처리도 테두리를 포함하여 자동으로 계산이 가능하기 때문에 PyAutoGUI.screenshot()를 사용하여 문제를 해결함.

**2. 주제를 입력하지 않고 다음 창으로 넘어갔을 때 되돌아갈 방법이 없음**

frame5에 "이전" 버튼을 추가하고 frame4로 돌아갈 수 있도록 함.

frame5를 숨기고 frame4를 다시 보여주기 위해 frame5_1.pack_forget(), frame5_2.pack_forget()를 사용하여 frame5_1과 frame5_2를 각각 숨기고, frame4()를 다시

호출할 수 있도록 수정함.

```
# <Import>


from tkinter import *
from tkinter import messagebox
from tkinter import colorchooser
from PIL import Image, ImageTk, ImageGrab
import json
import os
from datetime import datetime
import openai
from io import BytesIO
import requests
from googletrans import Translator
import random
import threading
from tkinter.ttk import Progressbar



# <함수>


# 1. 데이터 관리


def save_data(data):
    with open(data_file, 'w') as f:
        json.dump(data, f)


def load_data():
```

```
    if os.path.exists(data_file):
        with open(data_file, 'r') as f:
            return json.load(f)
    return {}

#----------------------------------------
def save_image():
    global current_time, user_image, image_count, img_path, user_folder, image_title
    user_folder = f"user_images/{user_name}"
    os.makedirs(user_folder, exist_ok=True)

    canvas_x = canvas.winfo_rootx()+170
    canvas_y = canvas.winfo_rooty()+100
    canvas_width = canvas.winfo_width()
    canvas_height = canvas.winfo_height()

    user_image = ImageGrab.grab(bbox=(canvas_x, canvas_y, canvas_x + canvas_width+50,
canvas_y + canvas_height))

    current_time = datetime.now().strftime("%Y%m%d_%H%M%S")
    formatted_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")  # 예: 2023-11-10
15:30:45

    image_count = len(os.listdir(user_folder)) + 1
    # 파일명에 제목과 날짜 추가
    img_path = f"{user_folder}/{image_count}_{image_title}_{current_time}.png"
    user_image.save(img_path)
```

```
def load_images():
    images = []
    # 사용자 폴더에서 이미지 파일 목록 가져오기
    for filename in os.listdir(user_folder):
        if filename.endswith(('.png', '.jpg', '.jpeg', '.gif')):  # 이미지 확장자 필터링
            img_path = os.path.join(user_folder, filename)
            img = Image.open(img_path)
            img = img.resize((200, 250))  # 원하는 크기로 조정
            img_tk = ImageTk.PhotoImage(img)  # Tkinter에서 사용할 수 있도록 변환
            # Extract the image title from the filename (the part before the first '_')
            image_title = filename.split('_')[1]  # Assuming the title is stored after the first '_'
            images.append((img_path, img_tk, image_title))  # Include title in the tuple
    return images

#----------------------------------------
def save():
    data['name'] = frame1_name.get()
    data['first_run'] = False
    save_data(data)
    os.makedirs(f"user_images/{data['name']}", exist_ok=True)
    frame2()

def save_title():
        global image_title
        image_title = frame4_topic.get()  # 제목 저장
        frame5()  # 다음 단계로 이동
```

# 2. 그림판

```
# 그리기 시작할 때 호출되는 함수
def start_draw(event):
    global last_x, last_y
    last_x, last_y = event.x, event.y

# 마우스를 움직일 때 연속적으로 선을 그리는 함수
def paint(event):
    global last_x, last_y
    if last_x is not None and last_y is not None:
        # 이전 위치와 현재 위치를 연결하는 선 그리기
        canvas.create_line(last_x, last_y, event.x, event.y, fill=pen_color, width=pen_size)
    # 현재 위치를 마지막 위치로 업데이트
    last_x, last_y = event.x, event.y

# 마우스 버튼을 놓을 때 그리기를 종료하는 함수
def end_draw(event):
    global last_x, last_y
    last_x, last_y = None, None

def change_color(color):
    global pen_color
    pen_color = color

def choose_color():
```

```
    global pen_color
    color_code = colorchooser.askcolor(title="색상 선택")
    if color_code[1]:  # 색상이 선택되었는지 확인
        pen_color = color_code[1]

def size_up():
    global pen_size
    pen_size += 2

def size_down():
    global pen_size
    if pen_size>2:
        pen_size -= 2


# 3. API 관련 함수


def generate_image_from_topic(topic):
    if topic in image_cache:
        return image_cache[topic]  # 캐싱된 이미지 반환

    # 이미지 생성 과정
    translated_topic = translator.translate(topic, src='ko', dest='en').text
    styled_prompt = f"{random.choice(styles)} {translated_topic}"

    response = openai.Image.create(prompt=styled_prompt, n=1, size="256x256")
```

```
        image_url = response['data'][0]['url']
        img_response = requests.get(image_url)
        img_data = BytesIO(img_response.content)

        generated_image = Image.open(img_data)
        image_cache[topic] = generated_image  # 캐시에 저장
        return generated_image

def generate_image_async(topic, callback):
    def task():
        image = generate_image_from_topic(topic)
        callback(image)
    threading.Thread(target=task, daemon=True).start()

# DALL-E 모델을 통해 이미지를 생성하는 함수 (번역 포함)
def generate_image_from_topic(topic):
    try:
        # 주제를 영어로 번역
        translated_topic = translator.translate(topic, src='ko', dest='en').text

        # 무작위로 스타일 선택
        styles = [
            "a watercolor painting of",
            "a colorful illustration of",
            "a digital artwork of",
            "a vibrant cartoon drawing of",
            "an abstract art piece of",
```

```
            "an oil painting of"
        ]
        selected_style = random.choice(styles)

        # 스타일과 주제를 포함한 프롬프트
        styled_prompt = f"{selected_style} {translated_topic}"

        # DALL-E API를 사용해 이미지 생성
        response = openai.Image.create(prompt=styled_prompt, n=1, size="256x256")
        image_url = response['data'][0]['url']

        # 이미지 다운로드
        img_response = requests.get(image_url)
        img_data = BytesIO(img_response.content)
        return Image.open(img_data)

    except Exception as e:
        messagebox.showerror("이미지 생성 오류", f"이미지 생성 중 오류가 발생했습니다: {e}")
        return None

#----------------------------------------
def regenerate_image():
    """
    이미지 재생성 버튼 클릭 시 호출됩니다.
    """
    global img_label_0
```

```
    # "로딩 중" 메시지 표시
    img_label_0.config(text="이미지를 생성 중입니다...", image="")  # 이미지 대신 텍스트 표시

    def on_image_generated(image):
        if image:
            img_tk = ImageTk.PhotoImage(image)
            img_label_0.config(image=img_tk, text="")  # 생성된 이미지로 교체
            img_label_0.image = img_tk  # 참조 유지
        else:
            img_label_0.config(text="이미지를 생성할 수 없습니다.")  # 실패 메시지 표시

    # 비동기로 이미지 생성
    generate_image_async(image_title, on_image_generated)

#-------------------------------------------
def translate_text(topic):
    if topic in translation_cache:
        return translation_cache[topic]
    translated_topic = translator.translate(topic, src='ko', dest='en').text
    translation_cache[topic] = translated_topic
    return translated_topic

# 긍정적인 피드백을 생성하는 함수
def generate_positive_feedback():
    try:
        prompt_text = "이 작품에 대해 긍정적이고 격려하는 코멘트를 해줘. 친구처럼 편하게 말해줘!₩
```

```
            그림과 관련없는 얘기를 하면 안돼"
            response = openai.ChatCompletion.create(
                model="gpt-3.5-turbo",
                messages=[
                    {"role": "system", "content": "너는 친구처럼 긍정적이고 격려하는 피드백을 주는 예술 멘토야.₩
                     그림과 관련없는 얘기를 하면 안돼"},
                    {"role": "user", "content": prompt_text}
                ],
                max_tokens=150,  # 더 긴 피드백을 위해 max_tokens 값을 증가
                temperature=1.0
            )
            feedback_text = response.choices[0].message['content'].strip()
            return feedback_text
        except Exception as e:
            messagebox.showerror("피드백 생성 오류", f"피드백 생성 중 오류가 발생했습니다: {e}")
            return "피드백을 생성할 수 없습니다."


# 4. 갤러리

def create_image_grid(frame):
    images = load_images()  # 이미지 로드
    rows = (len(images) + 2) // 3  # 이미지 개수에 따라 행 수 계산 (3개씩 배치)
    cols = 3  # 한 행에 3개

    for r in range(rows):
```

```
        for c in range(cols):
            index = r * cols + c
            if index < len(images):  # 이미지가 있는 경우에만 표시
                img_path, img_tk, image_title = images[index]  # 경로, img_tk, 제목을 튜플로 받음
                img_btn = Button(frame, image=img_tk, command=lambda img_path=img_path, image_title=image_title: show_image(img_path, image_title))
                img_btn.image = img_tk  # Button에서 ImageTk 객체에 대한 참조 유지
                img_btn.grid(row=r, column=c, padx=5, pady=5)

def show_image(img_path, image_title):
    top = Toplevel()
    top.title("이미지 보기")

    try:
        # 파일 이름에서 날짜 부분 추출
        img_name = os.path.basename(img_path)
        parts = img_name.split('_')

        if len(parts) >= 3:
            title = parts[1]  # 제목 추출
            date_str = parts[2]  # 날짜 추출
            formatted_date = f"{date_str[:4]}-{date_str[4:6]}-{date_str[6:8]}"
        else:
            title = "제목 없음"
            formatted_date = "날짜 없음"

        # 이미지 로드 및 크기 조정
```

```
            img = Image.open(img_path)
            img = img.resize((400, 500), Image.LANCZOS)
            img_tk_resized = ImageTk.PhotoImage(img)

            img_label = Label(top, image=img_tk_resized)
            img_label.image = img_tk_resized
            img_label.pack()

            # 제목, 날짜, 시간 라벨 추가
            title_label = Label(top, text=f"제목: {title}", font=("나눔고딕OTF", 12, "bold"))
            title_label.pack()
            date_label = Label(top, text=f"날짜: {formatted_date}")
            date_label.pack()

        except IndexError:
            error_label = Label(top, text="정보를 찾을 수 없습니다.", fg="red")
            error_label.pack()

        close_button = Button(top, text="닫기", command=top.destroy)
        close_button.pack(pady=10)

        top.geometry("400x600")


# 5. 설정창

def create_set_btn(parent):
```

```
    set_btn = Button(parent, text="설정", fg="white", bg="black", command=open_set_win)
    set_btn.place(relx=0.025, rely=0.05, anchor='center')
    return set_btn

def open_set_win():
    set_win = Toplevel(win)
    set_win.title("설정")
    set_win.geometry("300x200")
    set_win.resizable(False, False)

    set_win_name = Label(set_win, text="이름 변경")
    set_win_name.place(relx=0.2, rely=0.3, anchor='center')

    new_name = Entry(set_win)
    new_name.place(relx=0.5, rely=0.3,width = 100, anchor='center')

    def resave_name():
        global user_name
        data['name'] = new_name.get()
        user_name = new_name.get()
        save_data(data)
        set_win.destroy()

    set_win_name_btn = Button(set_win, text="완료", fg="white", bg="black",
command=resave_name)
    set_win_name_btn.place(relx=0.8, rely=0.3, anchor='center')
```

```
# <프레임>


win = Tk()


win.geometry("1000x500")

win.title("그림 프로그램")

win.option_add("*Font","나눔고딕OTF 13")

win.resizable(False, False)


data_file = 'data.json'

data = load_data()


openai.api_key = "sk-proj-UMrIYuedGxOxKW8nLnj0ptVRrbNuQN-S1Unt1hwbs4TbHtfTMjz2vZmS4₩

-rvBOZhZgGmPT1cC_T3BlbkFJ5OUUOYp1Obte9IiYgkHg_F-
OgEkMIQknmQiv_a8wOlBtd7W3yKsBDyDbzCsnzT3zX9Pe5d3mgA"

translator = Translator()

translation_cache = {}

image_cache = {}

last_x, last_y = None, None




def frame1():

    global frame1_name

    frame1 = Frame(win, width=1000, height=500)

    frame1.place(x=0, y=0)
```

```
    frame1_text = Label(frame1, text = "안녕! 이름이 뭐야?")
    frame1_text.place(relx=0.5, rely=0.4, anchor='center')

    frame1_name = Entry(frame1)
    frame1_name.place(relx=0.5, rely=0.45, anchor='center')

    data['name'] = frame1_name.get()
    data['first_run'] = True

    save_data(data)

    frame1_btn = Button(frame1, text = "입력", fg = "white", bg = "black", command = lambda: save())
    frame1_btn.place(relx=0.5, rely=0.525, anchor='center')

def frame2():
    global user_name
    for widget in win.winfo_children():
        widget.destroy()

    data = load_data()

    frame2 = Frame(win, width=1000, height=500)
    frame2.place(x=0, y=0)

    user_name = data.get('name')
```

```
    frame2_text = Label(frame2, text = "반가워 %s!!₩n 나는 노스체라고 해.₩n 우리 같이 그림을 그려보자!"%user_name)
    frame2_text.place(relx=0.5, rely=0.4, anchor='center')

    frame2_btn1 = Button(frame2, text = "사용 방법",fg = "white", bg = "black",
                         width = 10, command = lambda: frame3())
    frame2_btn1.place(relx=0.43, rely=0.55, anchor='center')

    frame2_btn2 = Button(frame2, text = "사용 시작",fg = "white", bg = "black",
                         width = 10, command = lambda: frame4())
    frame2_btn2.place(relx=0.56, rely=0.55, anchor='center')


def frame2_2():

    global user_name
    data = load_data()

    frame2_2 = Frame(win, width=1000, height=500)
    frame2_2.place(x=0, y=0)

    user_name = data.get('name')
    frame2_2_text = Label(frame2_2, text = "안녕 %s!!₩n오늘도 같이 그림을 그려보자!"%user_name)
    frame2_2_text.place(relx=0.5, rely=0.4, anchor='center')

    frame2_2_btn1 = Button(frame2_2, text = "사용 방법",fg = "white", bg = "black",
                         width = 10, command = lambda: frame3())
```

```
    frame2_2_btn1.place(relx=0.43, rely=0.55, anchor='center')


    frame2_2_btn2 = Button(frame2_2, text = "사용 시작",fg = "white", bg = "black",
                           width = 10, command = lambda: frame4())
    frame2_2_btn2.place(relx=0.56, rely=0.55, anchor='center')



def frame3():
    for widget in win.winfo_children():
        widget.destroy()


    frame3 = Frame(win, width=1000, height=500)
    frame3.place(x=0, y=0)


    frame3_text = Label(frame3, text = """~사용 방법~₩n₩n
    1.그리고 싶은 주제를 입력한다!₩n
    2.노스체가 그려준 그림을 참고해서 나만의 그림을 그려보자!₩n
    3.그림을 저장하고 갤러리를 확인해보자!₩n₩n
    ~아주 작은 팁~₩n₩n
    노스체는 <Rogers의 무조건적 긍정적 수용의 원리>를 바탕으로₩n
    %s의 모든 경험을 있는 그대로 수용하고 존중해!₩n
    그림을 그리다가 생기는 실수는 부정적인 경험이지만, 나쁜 경험도 때로는 받아들일 필요가 있지.₩n
    '지우개 없는 그림판'을 통해 내면을 솔직하게 표현하고 자기 검열로부터 자유로운 미술 시간을 함께 만들어보자!₩n
    노스체는 %s의 어떤 활동이나 감정도 이해해^^₩n
    이제는 나에게 솔직해질 시간!"""%(user_name,user_name), font=("나눔고딕OTF", 10))
```

```
    frame3_text.place(relx=0.5, rely=0.4, anchor='center')

    frame3_btn = Button(frame3, text = "사용 시작",fg = "white", bg = "black",
                                width = 10, command = lambda: frame4())
    frame3_btn.place(relx=0.5, rely=0.9, anchor='center')
    create_set_btn(frame3)

def frame4():
    global frame4_topic, image_title
    for widget in win.winfo_children():
        widget.destroy()

    frame4 = Frame(win, width=1000, height=500)
    frame4.place(x=0, y=0)

    frame4_text = Label(frame4, text="오늘은 무슨 그림을 그릴까?")
    frame4_text.place(relx=0.5, rely=0.4, anchor='center')

    frame4_topic = Entry(frame4)
    frame4_topic.place(relx=0.5, rely=0.45, anchor='center')

    frame4_btn = Button(frame4, text="입력", fg="white", bg="black", command=save_title)
    frame4_btn.place(relx=0.5, rely=0.525, anchor='center')
    create_set_btn(frame4)

def frame5():
```

```
global canvas, pen_color, pen_size, img_label_0

pen_color = "black"
pen_size = 6
for widget in win.winfo_children():
    widget.destroy()

frame5_1 = Frame(win, width=500, height=500, bg = "lightblue")
frame5_1.pack(side=LEFT)

frame5_2 = Frame(win, width=500, height=500,
highlightbackground = "black", highlightthickness = 1)
frame5_2.pack(side=RIGHT)

frame5_1_text = Label(frame5_1, text = "여기는 내가 그린 그림이야")
frame5_1_text.place(relx=0.5, rely=0.1, anchor='center')

if image_title in image_cache:
    generated_image = image_cache[image_title]
else:
    generated_image = generate_image_from_topic(image_title)

if generated_image:
    img_tk_0 = ImageTk.PhotoImage(generated_image)
    img_label_0 = Label(frame5_1, image=img_tk_0)
    img_label_0.image = img_tk_0
    img_label_0.place(relx=0.5, rely=0.5, anchor='center')
```

```
    else:
        Label(frame5_1, text="이미지를 생성할 수 없습니다.").place(relx=0.5, rely=0.5, anchor='center')


    frame5_1_btn = Button(frame5_1, text="재생성", width=5, fg="white", bg="black", command=regenerate_image)
    frame5_1_btn.place(relx=0.1, rely=0.9, anchor='center')

    canvas = Canvas(frame5_2,width=400, height=500)
    canvas.pack(side = RIGHT)

    canvas.bind("<ButtonPress-1>", start_draw)      # 마우스 버튼 눌림
    canvas.bind("<B1-Motion>", paint)               # 마우스 드래그
    canvas.bind("<ButtonRelease-1>", end_draw)

    option_frame = Frame(frame5_2, width = 60, height = 500, bg = 'gray')
    option_frame.pack(side=LEFT)

    btn_black = Button(option_frame, width = 5, bg = "black", command = lambda: change_color("black"))
    btn_black.place(x=0, y=0)

    btn_red = Button(option_frame, width = 5, bg = "red", command = lambda: change_color("red"))
    btn_red.place(x=0, y=40)

    btn_blue = Button(option_frame, width = 5, bg = "blue", command = lambda:
```

```
change_color("blue"))
    btn_blue.place(x=0, y=80)

    #-----------------------
    btn_colors = Button(option_frame, width = 5, text = "다른색", font = ("나눔고딕 OTF",'12'),command = choose_color) #색상창함수: 일단 아무거나 넣어둠
    btn_colors.place(x=0, y=120)

    #---------------------------
    btn_size_up = Button(option_frame, text = "굵게", width = 5, font = ('나눔고딕OTF','12'), command = size_up)
    btn_size_up.place(x=0, y = 160)

    btn_size_down = Button(option_frame, text = "가늘게", width = 5, font = ('나눔고딕OTF','12'), command = size_down)
    btn_size_down.place(x=0, y = 200)

    frame5_2_btn = Button(option_frame, text = "✓", fg = "white", bg = "black", command=lambda: [save_image(), frame6()])
    frame5_2_btn.place(relx=0.5, rely=0.9, anchor='center')

    # "이전" 버튼 추가하여 frame4로 돌아가기
    previous_btn = Button(frame5_2, text="이전", fg="white", bg="black", command=lambda: [frame5_1.pack_forget(), frame5_2.pack_forget(), frame4()])
    previous_btn.place(x=0,y=360)

def frame6():
    global user_image
```

```
for widget in win.winfo_children():
    widget.destroy()

frame6 = Frame(win, width=1000, height=500)
frame6.place(x=0, y=0)

user_image = user_image.resize((300, 300))
user_image_tk = ImageTk.PhotoImage(user_image)

user_image_print = Label(frame6, image=user_image_tk)
user_image_print.place(relx=0.5, rely=0.3, anchor='center')
user_image_print.image = user_image_tk

frame6_btn1 = Button(frame6, text="갤러리", fg="white", bg="black",
                     width=10, command=lambda: gallery_frame())
frame6_btn1.place(relx=0.38, rely=0.7, anchor='center')

frame6_btn2 = Button(frame6, text="더 그리기", fg="white", bg="black",
                     width=10, command=lambda: frame4())
frame6_btn2.place(relx=0.5, rely=0.7, anchor='center')

frame6_btn3 = Button(frame6, text="종료", fg="white", bg="black",
                     width=10, command=lambda: win.destroy())
frame6_btn3.place(relx=0.62, rely=0.7, anchor='center')

feedback_text = generate_positive_feedback()
feedback_label = Label(frame6, text=feedback_text, wraplength=600, justify="center", font=("
```

```
나눔고딕OTF", 12))
    feedback_label.place(relx=0.5, rely=0.85, anchor="center")

def gallery_frame():
    for widget in win.winfo_children():
        widget.destroy()

    canvas2 = Canvas(win)
    scrollbar = Scrollbar(win, orient="vertical", command=canvas2.yview)
    scrollable_frame = Frame(canvas2)

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas2.configure(scrollregion=canvas2.bbox("all")))

    canvas2.create_window((0, 0), window=scrollable_frame, anchor="nw")

    scrollbar.pack(side="right", fill="y")
    canvas2.pack(side="left", fill="both", expand=True)
    canvas2.config(yscrollcommand=scrollbar.set)

    create_image_grid(scrollable_frame)

    gframe_btn1 = Button(scrollable_frame, text="더 그리기", fg="white", bg="black", width=10,
command=lambda: frame4())
    gframe_btn1.grid(row=0, column=3, padx=5, pady=5)
```

```
    gframe_btn2 = Button(scrollable_frame, text="종료", fg="white", bg="black", width=10, command=lambda: win.destroy())

    gframe_btn2.grid(row=1, column=3, padx=5, pady=5)



# <실행>


if data.get('first_run', True):

    frame1()

else:

    frame2_2()



win.mainloop()
```

# <맡은 역할>

| | |
|---|---|
| 김채현 | -노스체 설명글 추가<br>-시연 및 소개 영상 제작 |
| 도건우 | -최종보고서 만듦 (보고서 순서 고려, 취합 및 병합) |
| 양정현 | -코드 순서대로 정리<br>-그림 판 작동 방식 수정 (관련 함수 변경 및 추가)<br>-이름 변경 설정 창 추가<br>-사용 설명 버튼 추가<br>-화면 작동 구조 수정<br>-프로그램의 가치 달성을 효율적으로 수행하기 위한 전반적인 수정<br>(1). 친구 같은 그림 프로그램, API 요청 프롬프트 수정<br>(2). 사용 방법 글 수정 및 추가<br>-PPT 제작 (기획 의도, 기대 효과) 및 PPT 디자인 |
| 유서현 | - open api 관련 7가지 코드 수정 및 7가지 코드 수정사항 설명 작성<br>- 라이브러리 설치 가이드 문서 작성<br>- Open api 구매<br>- 의견취합 및 역할분담<br>- 구글폼 및 제출물 제출 |
| 유서진 | - 캔버스 캡처 제대로 되도록 수정<br>- 이미지 드로잉 창에서 이전 화면으로 넘어갈 수 있도록 하는 버튼 생성<br>- 피피티 제작(사용방법, 활용방안) |

| | |
|---|---|
| | |